PARENT ABSENCE NOTIFICATION

Software Design Document (SDD)

Enhancement of the SAMS Attendance Module

Prepared in alignment with the Phase-2 (Software Design) guideline

Document Item	Details
Project / System	SAMS – School Attendance Management System
Module	Attendance / Notifications
Document Type	Feature Enhancement Software Design Document
Version	1.0
Prepared By	Software Design Team
Date	2 November 2026
Status	Completed Design Baseline - Pending Stakeholder Approval

Table of Contents

1. Document Control 4
1.1 Finalization Baseline and Decisions 4
2. Purpose and Scope 4
2.1 Purpose 4
2.2 In Scope 5
2.3 Out of Scope 5
3. Existing Attendance Feature Overview 5
4. Proposed Feature Enhancements 5
4.1 Functional Requirements 6
5. Interface Design (UI & UX) 6
5.1 Parent – Notification and Reason Form 7
5.2 Parent – Portal 7
5.3 Teacher and Office Staff 7
5.4 Administrator 7
5.5 Baseline Wireframe Specifications 7
6. Architectural Design 8
6.1 Component Responsibilities 8
7. Data Flow Design 9
7.1 Key Data Flows 10
8. Database Design 10
8.1 Proposed Tables / Changes 10
8.2 Field Definitions 11
8.3 Data Integrity and Performance 12
9. Program and API Design 12
9.1 API Contracts 12
9.2 Notification Processing Rules 13
9.3 Pseudocode – Process Notification Events 14
9.4 UML Sequence Diagrams 15
9.4.1 Absence Notification 15
9.4.2 Parent Submits Absence Reason 16
10. Software Report / Analytics Design 16
11. Security, Privacy, and Non-Functional Design 16
11.1 Security & Privacy 16
11.2 Non-Functional Requirements 17
12. Impact Analysis and Migration 17
12.1 Deployment Approach 17
12.2 Rollback Considerations 18
12.3 Deployment Environments and Release Gates 18
13. Test Design 18
13.1 Regression Areas 19

14. Acceptance Criteria..... 19
15. Requirement-to-Design Traceability..... 19
16. Finalized Implementation Decisions..... 20
Appendix A – Completion and Approval Checklist..... 20
Appendix B – Review and Approval Record..... 20

1. Document Control

Executive assessment: The Release 1 design covers every Must and Should requirement in the BRD (BR-01 to BR-11) and is traceable to tests. Production approval should depend on cleaning up missing student–parent links, confirming SendGrid sending limits, and naming an owner for the 11:00 follow-up list. **Decision requested:** approve F-01 to F-08 for implementation, subject to the release gates in Section 12.3. The main delivery risks are incomplete guardian data, email deliverability at the 08:30 peak, and correct handling of corrections made shortly after cut-off.

This document describes the software design for adding parent absence notifications to the existing SAMS attendance module. It combines architectural and detailed design so the enhancement can be reviewed before implementation and maintained after release.

Version	Date	Author	Change
1.0	2 November 2026	Software Design Team	First complete design baseline produced from BRD v1.0 (20 October 2026).

1.1 Finalization Baseline and Decisions

The BRD leaves several design choices open. To make this SDD implementation-ready without presenting those choices as decided facts, this version adopts the following explicit baseline. Any later change must be recorded as a versioned design change before production release.

- Release 1 includes features F-01 to F-08 defined in Section 4. SMS and Telegram remain out of scope (BRD §6).
- Notification processing runs as a scheduled job every minute; the first absence run for a school is at its configured cut-off time (default 08:30, school time zone Asia/Phnom_Penh).
- A correction window of 30 minutes after a notification is sent applies; changing Absent to Present within this window sends one correction message.
- Release 1 channels are email (SendGrid) and in-app (parent portal). Each parent must keep at least one channel enabled.
- Absence-reason links are signed, single-student tokens valid for 72 hours; parents who are logged in can also answer from the portal.
- Attachments for absence reasons are limited to PDF, JPG and PNG up to 5 MB each.
- Notification logs and absence reasons are kept for the current academic year plus 90 days, then deleted; monthly aggregates are kept for 3 years.
- Templates are stored per event type and language (English, Khmer); the parent's preferred language decides which is used, with English as fallback.

2. Purpose and Scope

2.1 Purpose

The purpose of this design is to notify parents promptly when a student is absent or late, let them submit a reason, and give staff a reliable record, without changing how teachers take attendance. It defines the interface, components, data flows, database changes, APIs, security controls and tests needed.

2.2 In Scope

- Automatic absence and late notifications after a configurable cut-off time.
- Correction messages when attendance is changed to Present within the correction window.
- Email and in-app channels, with parent channel preferences.
- Parent absence-reason submission with optional attachment.
- Staff view of reasons and an office follow-up list.
- Administrator configuration of cut-off times, late notifications and bilingual templates.
- Notification log and monthly absence/notification report.

2.3 Out of Scope

- SMS, Telegram or other messaging channels (phase 2).
- Automatic absence prediction or risk scoring.
- Changes to the teacher attendance-taking screens or QR check-in flow.
- Fixing guardian data in source systems beyond the data-quality report in Section 12.

3. Existing Attendance Feature Overview

SAMS already records daily attendance through the web screen and QR-code check-in. All attendance writes go through a single attendance service, which records the source of each change. Office staff currently phone parents manually. The enhancement reuses the attendance service as the trigger point and adds a notification subsystem.

Area	Existing Behavior (Baseline)	Enhancement Impact
Attendance recording	Teachers mark Present/Absent/Late via web or QR; one attendance service writes records.	Attendance service emits an event on every status change; no UI change for teachers.
Parent contact	Office phones parents manually each morning; no record kept.	Automatic email and in-app notifications with a full delivery log.
Parent portal	Parents can sign in and view basic student information.	Adds notification inbox, absence-reason form and channel preferences.
Staff screens	Attendance list per class.	Shows absence reason and response status; new office follow-up list.
Administration	User, class and student management.	Adds notification settings, cut-off times and template editor.
Email	SendGrid used for account emails.	Reused for notifications with delivery-status webhooks.

4. Proposed Feature Enhancements

ID	Feature	Design Intent
F-01	Absence notification	Notify every linked parent when a student is marked Absent after the cut-off time.

ID	Feature	Design Intent
F-02	Late notification	Optionally notify parents when a student is marked Late (school setting).
F-03	Correction message	Send a correction when a record changes to Present within the correction window.
F-04	Absence reason	Parents submit a reason and optional attachment from the notification link or portal.
F-05	Staff reason view & follow-up	Teachers and office see reasons; office sees unanswered absences after 11:00.
F-06	Channel preferences	Parents choose email and/or in-app; at least one must stay on.
F-07	Admin configuration & templates	Admins set cut-off times, enable late notices and edit English/Khmer templates.
F-08	Notification log & reporting	Log every send and delivery status; monthly report by class.

4.1 Functional Requirements

ID	Requirement
FR-01	An Absent status that exists at or after the school cut-off time triggers one notification per linked parent per student per day (BR-01).
FR-02	Late notifications are sent only when the school setting late_notify_enabled is true (BR-02).
FR-03	Each notification is delivered through every channel the parent has enabled; in-app is always created as the record of the message (BR-03, BR-09).
FR-04	If a status changes from Absent/Late to Present within 30 minutes of the first notification, exactly one correction message is sent; later changes are logged only (BR-04).
FR-05	A parent can submit one reason per student per absence date; resubmission replaces the earlier reason and keeps a history (BR-05).
FR-06	Reasons and response status appear on the teacher class list and the office follow-up list (BR-06, BR-07).
FR-07	The follow-up list shows absences with no reason and no successful delivery, or no reason by 11:00 school time (BR-07).
FR-08	Only administrators can change cut-off times, late settings and templates; template changes are versioned (BR-08).
FR-09	Parents cannot disable all channels; the API rejects a preference update that would leave none enabled (BR-09).
FR-10	Every send attempt and delivery-status update is stored in the notification log (BR-10).
FR-11	Students without a linked parent are listed on the follow-up list as "No guardian contact" instead of failing silently.

5. Interface Design (UI & UX)

New screens follow the existing SAMS look and components. Parent screens are mobile-first because most parents open notifications on a phone. Staff screens add information to existing lists rather than creating new navigation where possible.

5.1 Parent – Notification and Reason Form

- Email contains student name, class, date, status (Absent/Late), the school phone number and a "Submit reason" button.
- The reason form opens from the signed link without a password; it shows the student, date and status read-only.
- Reason options: Sick, Family matter, Other; free-text up to 500 characters; optional attachment.
- After submission, a confirmation screen and an in-app confirmation are shown.
- All text is available in English and Khmer according to the parent's language setting.

5.2 Parent – Portal

- Notification inbox listing recent messages with unread markers.
- Notification preferences page with email and in-app toggles; the last enabled toggle cannot be switched off.

5.3 Teacher and Office Staff

- Class attendance list shows a reason icon and response status next to Absent/Late students.
- Office follow-up list filtered by date, class and reason status, with a "Mark contacted" action and note.

5.4 Administrator

- Notification settings per school: cut-off time, late notifications on/off, follow-up time (default 11:00).
- Template editor per event (Absence, Late, Correction) and language, with preview and placeholder list.

5.5 Baseline Wireframe Specifications

Screen	Primary Inputs / Controls	Primary Outputs
Reason Form	Reason type, text, attachment, submit	Confirmation, saved reason
Parent Inbox	Open message, mark read	Message list with status
Preferences	Email toggle, in-app toggle, language	Saved preferences / validation message
Office Follow-up	Date, class, status filters, mark contacted	Paginated list of unanswered absences
Template Editor	Event, language, subject, body	Preview and saved template version

Region	Baseline Layout / Behavior
Reason form header	School name and logo, student name, class, date and status badge.
Reason form body	Radio buttons for reason type, text area with character counter, attachment picker with size hint.
Reason form footer	Submit button full width on mobile; error messages shown above the button.
Follow-up list toolbar	Date picker (default today), class filter, status filter, export CSV.
Follow-up list rows	Student, class, status, parent contacts, delivery status, reason status, actions.

Region	Baseline Layout / Behavior
Empty / error states	Explain when there are no unanswered absences or when data could not load, with retry.

6. Architectural Design

The enhancement adds a Notification module inside the existing SAMS backend. The attendance service publishes status-change events to an outbox table in the same transaction as the attendance write. A scheduled Notification Worker reads due events, applies the rules and sends messages through channel adapters. This keeps teacher attendance fast and makes notifications reliable even if email is slow.

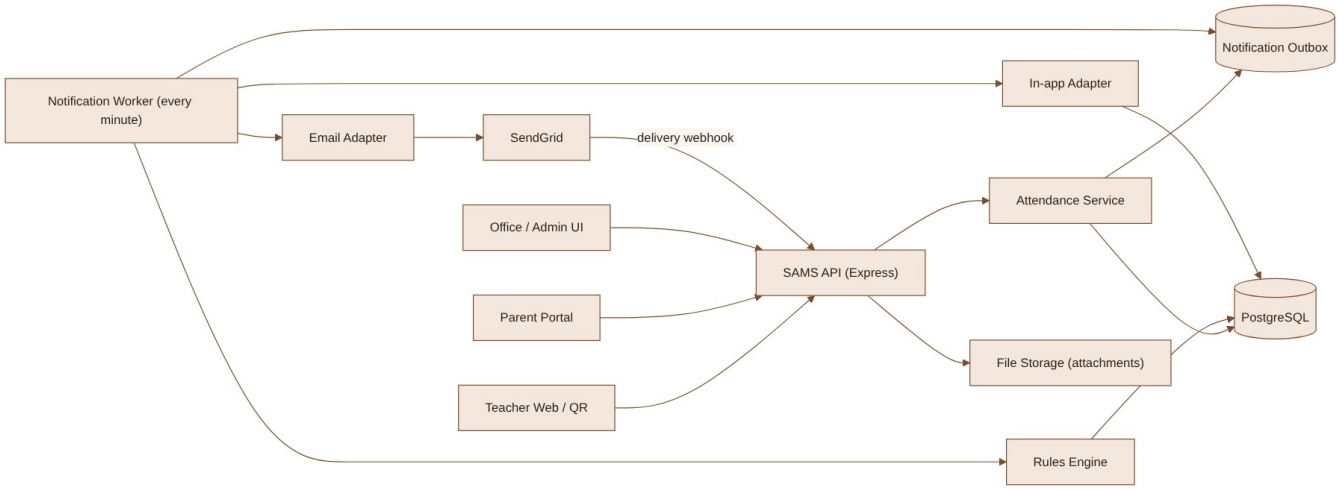


Figure 1. Proposed notification architecture

6.1 Component Responsibilities

Component	Responsibility
Attendance Service	Existing single write path for attendance; now also writes an outbox event on status change.
Notification Outbox	Durable queue table of attendance events waiting to be processed.
Notification Worker	Runs every minute; picks due events with row locking, calls rules, sends and logs.
Rules Engine	Checks cut-off time, late setting, correction window, deduplication and guardian links.
Channel Adapters	Email (SendGrid) and in-app; render templates and return a provider message ID.
Reason Service	Validates signed tokens, stores reasons and attachments, records history.
Follow-up Service	Builds the office follow-up list and records "contacted" actions.
Webhook Endpoint	Receives SendGrid delivery events and updates the notification log.

7. Data Flow Design

The Level-1 data flow below shows how teachers, parents, staff and external email interact with the notification processes and data stores.

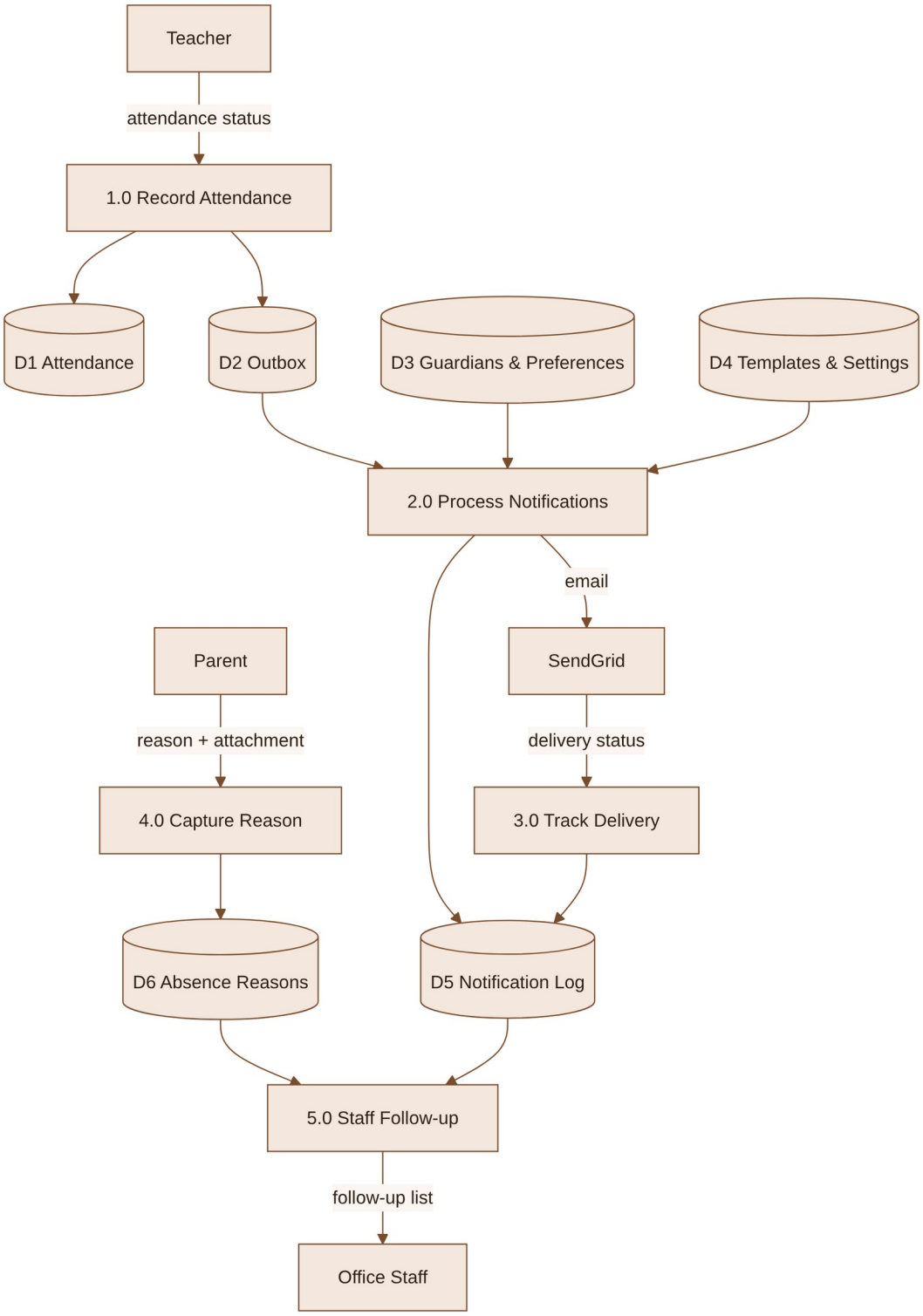


Figure 2. Level-1 Data Flow Diagram for parent absence notifications

7.1 Key Data Flows

- A teacher marks attendance; the attendance service writes the record and an outbox event in one transaction.
- The worker reads due events, loads guardians, preferences, settings and templates, and sends messages.
- SendGrid reports delivered, bounced or dropped status back through the webhook into the notification log.
- A parent opens the signed link, submits a reason, and the reason becomes visible to staff immediately.
- At the follow-up time, the follow-up service combines attendance, log and reason data for the office.

8. Database Design

The design adds tables to the existing PostgreSQL schema and reuses the existing students, classes, users and attendance tables. All new tables use UUID primary keys and timestamps in UTC.

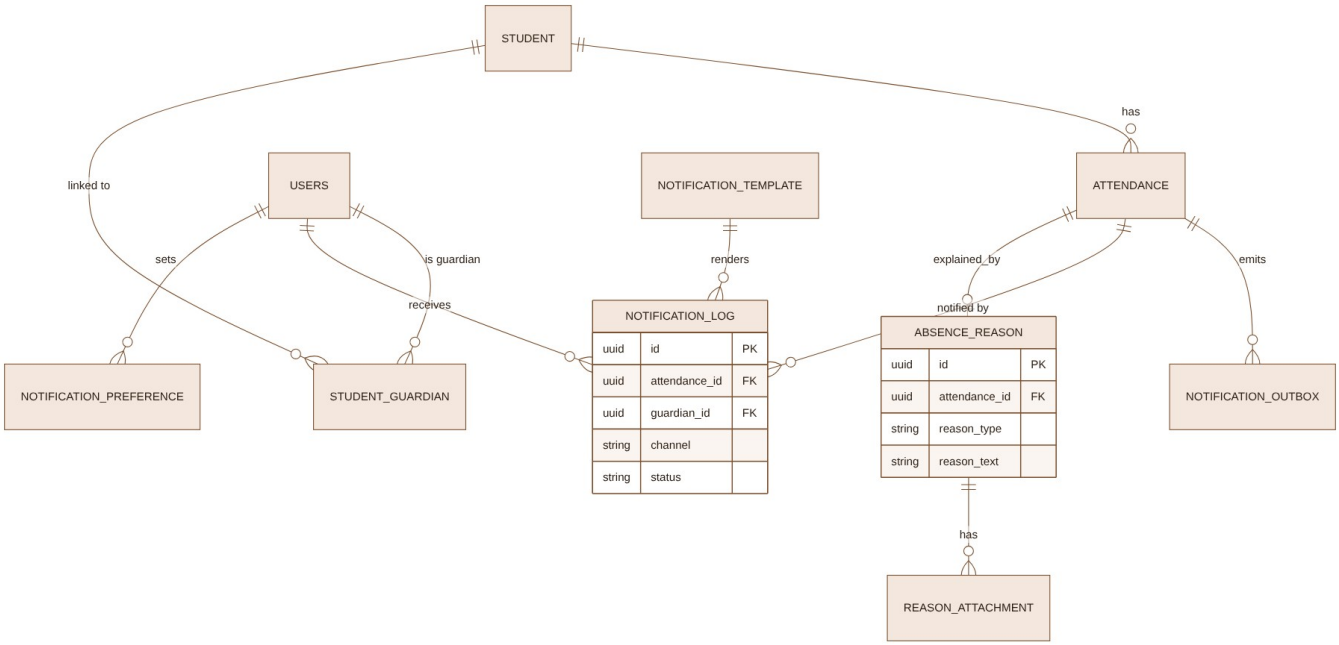


Figure 3. Conceptual Entity Relationship Diagram

8.1 Proposed Tables / Changes

Table	Type	Design Notes
ATTENDANCE	Existing	No column changes; status changes now also insert an outbox row in the same transaction.
STUDENT_GUARDIAN	Existing / Modify	Add is_primary and receives_notifications flags; UNIQUE(student_id, guardian_id).
NOTIFICATION_OUTBOX	New	Pending attendance events with due_at and processed_at; index on (processed_at, due_at).
NOTIFICATION_LOG	New	One row per message per channel per guardian; UNIQUE(attendance_id, guardian_id, event_type, channel).

Table	Type	Design Notes
NOTIFICATION_PREFERENCE	New	One row per guardian: email_enabled, in_app_enabled, language; CHECK at least one channel on.
NOTIFICATION_TEMPLATE	New	Versioned templates by school, event_type and language; only one active version.
NOTIFICATION_SETTING	New	Per school: cutoff_time, late_notify_enabled, followup_time, correction_window_min.
ABSENCE_REASON	New	Current reason per attendance record; history kept in ABSENCE_REASON_HISTORY.
REASON_ATTACHMENT	New	File metadata; files stored outside the database.
FOLLOWUP_ACTION	New	Office "contacted" actions with staff user, time and note.

8.2 Field Definitions

Table	Field	Type	Constraint / Index	Purpose
NOTIFICATION_OUTBOX	id	UUID	PK	Event identifier
NOTIFICATION_OUTBOX	attendance_id	UUID	FK, NOT NULL	Source attendance record
NOTIFICATION_OUTBOX	new_status	VARCHAR(10)	NOT NULL	ABSENT, LATE or PRESENT
NOTIFICATION_OUTBOX	due_at	TIMESTAMPTZ	NOT NULL, indexed	Earliest processing time (cut-off)
NOTIFICATION_OUTBOX	processed_at	TIMESTAMPTZ	NULL, indexed	Set when processed
NOTIFICATION_LOG	event_type	VARCHAR(12)	NOT NULL	ABSENCE, LATE or CORRECTION
NOTIFICATION_LOG	channel	VARCHAR(10)	NOT NULL	EMAIL or IN_APP
NOTIFICATION_LOG	status	VARCHAR(12)	NOT NULL, indexed	QUEUED, SENT, DELIVERED, BOUNCED, FAILED, READ
NOTIFICATION_LOG	provider_message_id	VARCHAR(100)	NULL, indexed	SendGrid message ID for webhooks
NOTIFICATION_LOG	sent_at	TIMESTAMPTZ	NULL	Time the message was sent
NOTIFICATION_PREFERENCE	email_enabled	BOOLEAN	DEFAULT true	Email channel opt-in
NOTIFICATION_PREFERENCE	in_app_enabled	BOOLEAN	DEFAULT true	In-app channel opt-in
NOTIFICATION_PREFERENCE	language	VARCHAR(5)	DEFAULT 'en'	en or km
ABSENCE_REASON	reason_type	VARCHAR(12)	NOT NULL	SICK, FAMILY or OTHER
ABSENCE_REASON	reason_text	VARCHAR(500)	NULL	Parent explanation

Table	Field	Type	Constraint / Index	Purpose
NOTIFICATION_SETTING	cutoff_time	TIME	DEFAULT 08:30	School-local absence cut-off

8.3 Data Integrity and Performance

- Write the attendance change and the outbox event in one database transaction so no event is lost.
- The unique constraint on NOTIFICATION_LOG prevents duplicate notifications if the worker retries.
- The worker selects outbox rows with FOR UPDATE SKIP LOCKED so two worker instances never process the same event.
- Guardian deletion cascades to preferences; notification logs keep the guardian ID but personal contact data is not copied into the log.
- Logs and reasons older than the academic year plus 90 days are deleted by a nightly job after monthly aggregates are stored.

9. Program and API Design

All endpoints are added to the existing Express API, use the existing JWT authentication and role checks, and are documented in Swagger.

9.1 API Contracts

Method	Endpoint	Input	Output / Behavior
GET	/api/parent/notifications	page, pageSize	Parent inbox, newest first; parent role required.
PATCH	/api/parent/notifications/{id}/read	–	Marks an in-app message as read.
GET/PUT	/api/parent/notification-preferences	email_enabled, in_app_enabled, language	Read/update preferences; 422 if no channel enabled.
GET	/api/absence-reasons/form?token=	Signed token	Student, date and status for the reason form; 410 if expired.
POST	/api/absence-reasons	token or session, reason_type, reason_text, attachments	Creates or replaces the reason; returns saved reason.
GET	/api/attendance/{classId}/reasons?date=	Class, date	Reasons and response status for a class; teacher/office role.
GET	/api/office/follow-ups	date, classId, status, page	Unanswered absences; office role.
POST	/api/office/follow-ups/{attendanceId}/ contacted	note	Records office contact.

Method	Endpoint	Input	Output / Behavior
GET/PUT	/api/admin/notification-settings	cutoff_time, late_notify_enabled, followup_time	School settings; admin role.
GET/POST	/api/admin/notification-templates	event_type, language, subject, body	List or create a new template version; admin role.
POST	/api/webhooks/sendgrid	SendGrid event batch (signed)	Updates delivery status; signature verified.
GET	/api/reports/absence-notifications?month=	Month, classId	Monthly report data; admin/head role.

9.2 Notification Processing Rules

1. Select outbox events where processed_at is null and due_at <= now, locking rows with SKIP LOCKED (batch of 200).
2. Load the current attendance status; if it no longer matches the event, mark the event processed and stop.
3. For ABSENT, require that the time is at or after the school cut-off; for LATE, require late_notify_enabled.
4. For PRESENT, send a CORRECTION only if an ABSENCE/LATE notification was sent within the correction window.
5. Load linked guardians with receives_notifications = true; if none, add the student to the follow-up list as "No guardian contact".
6. For each guardian and enabled channel, insert a QUEUED log row (skipping duplicates), render the template and send.
7. Update the log with SENT or FAILED; retry FAILED email up to 3 times with 1, 5 and 15 minute back-off.
8. Mark the outbox event processed.

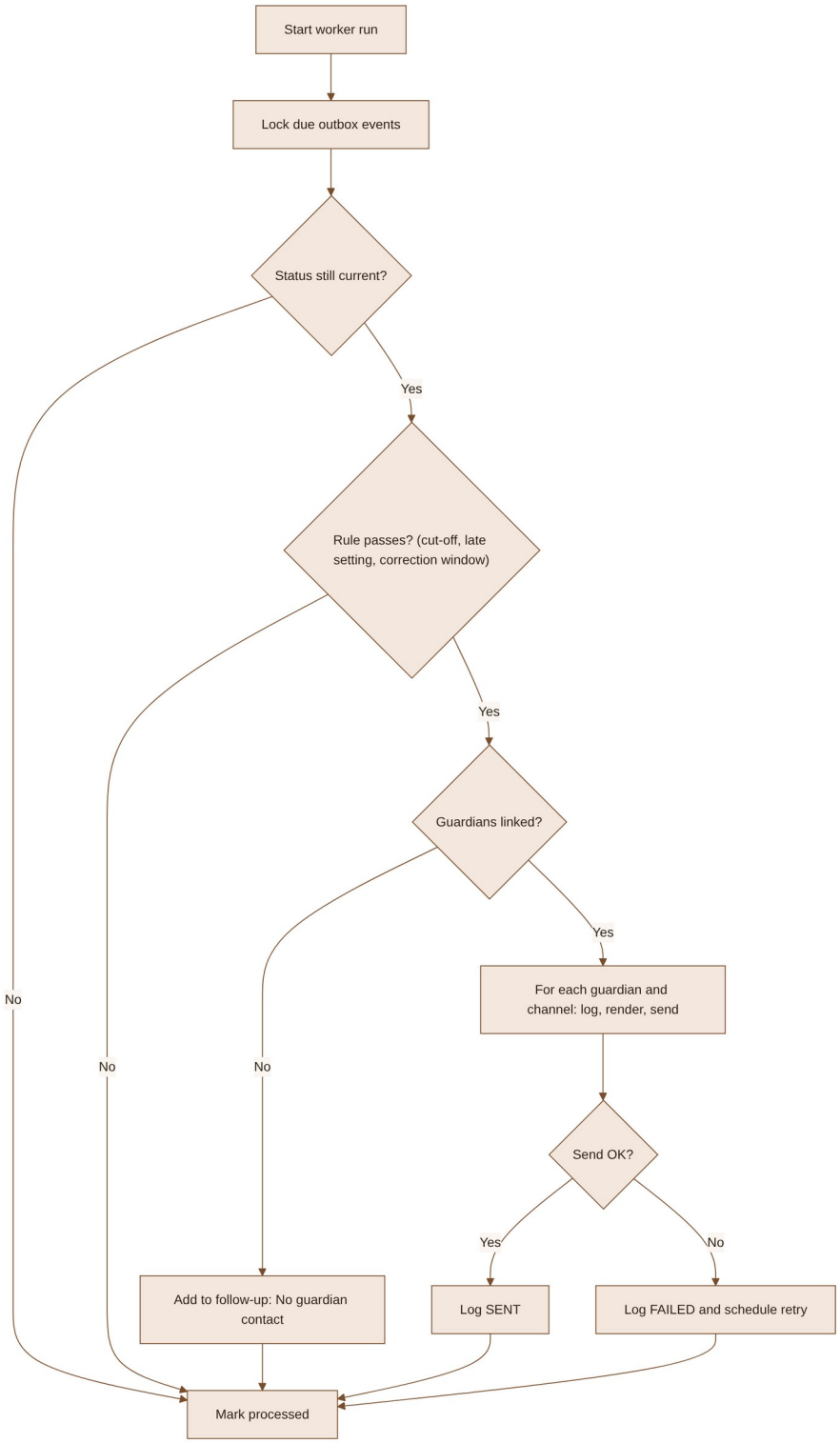


Figure 4. Notification worker program logic

9.3 Pseudocode – Process Notification Events

```
function processDueEvents(now):
    events = outbox.lockDue(now, limit = 200)        // FOR UPDATE SKIP LOCKED
    for event in events:
        record = attendance.get(event.attendanceId)
        if record.status != event.newStatus:
            outbox.markProcessed(event); continue
```

```
        settings = settings.forSchool(record.schoolId)
        type = rules.eventType(event, record, settings, now)   // ABSENCE | LATE |
CORRECTION | null
        if type is null:
            outbox.markProcessed(event); continue

        guardians = guardians.forStudent(record.studentId)
        if guardians is empty:
            followUps.flag(record, "NO_GUARDIAN_CONTACT")
        for g in guardians:
            for channel in preferences.enabledChannels(g):
                if log.insertIfAbsent(record, g, type, channel):
                    message = templates.render(type, g.language, record)
                    result = adapters[channel].send(g, message)
                    log.update(result)
        outbox.markProcessed(event)
```

9.4 UML Sequence Diagrams

The sequence diagrams show the order of calls for the two most important interactions and where authorization and validation happen.

9.4.1 Absence Notification

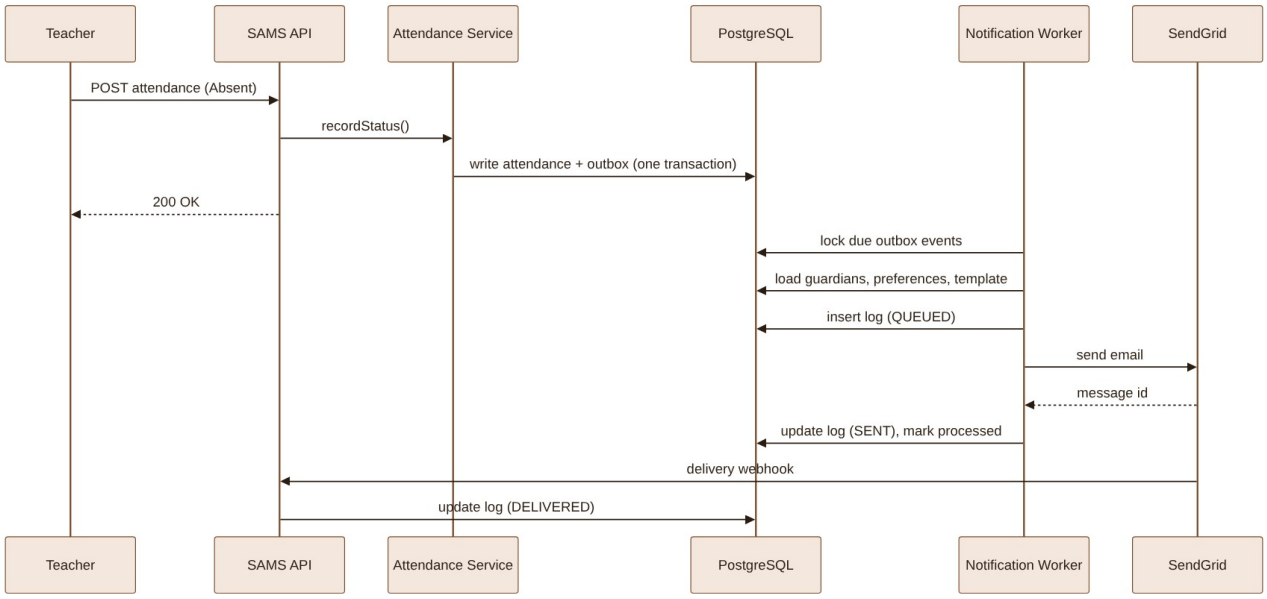


Figure 5. UML sequence diagram – Absence notification

Design note: the teacher request returns as soon as the attendance and outbox rows are written. Email sending happens in the worker, so a slow or failing email provider never blocks attendance taking.

9.4.2 Parent Submits Absence Reason

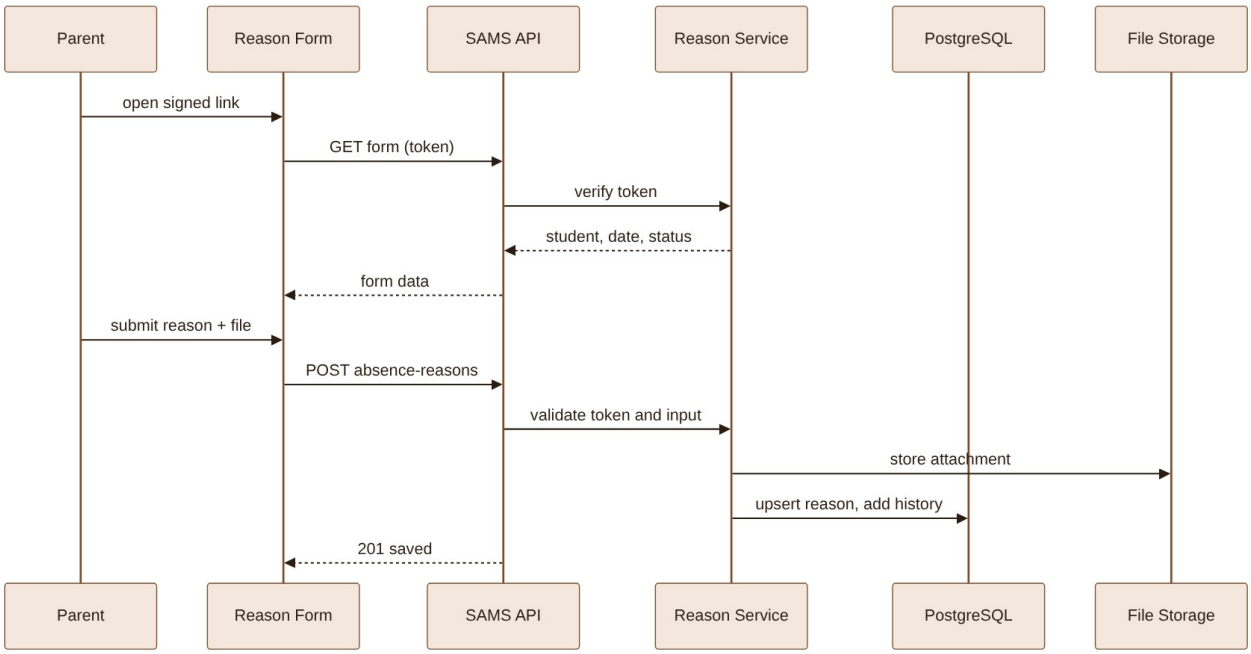


Figure 6. UML sequence diagram – Parent submits absence reason

Design note: the token identifies exactly one student and date and expires after 72 hours. It never grants access to other students or portal data.

10. Software Report / Analytics Design

Reporting supports the success measures in the BRD while keeping personal data to a minimum. Reports use monthly aggregates and are available only to administrators and the Head of Student Affairs.

Metric / Report	Definition	Possible Use
Absence count	Absent and Late records by class and month.	Identify classes with rising absence.
Delivery rate	Delivered ÷ sent notifications by channel.	Check email quality and guardian data.
Time to notify	Minutes from cut-off to SENT (p50, p95).	Measure the 10-minute target.
Parent response rate	Absences with a reason ÷ absences, same day.	Measure the 60% target.
Office follow-ups	Follow-up items and contacted count per day.	Measure reduction in manual calls.

Reports can be viewed on screen or exported as CSV. Exports exclude parent contact details.

11. Security, Privacy, and Non-Functional Design

11.1 Security & Privacy

- All staff and parent endpoints use existing JWT authentication with role checks on the server.
- Parents can only see notifications and students linked to their own account.

- Reason tokens are HMAC-signed, scoped to one student and date, expire after 72 hours and are rate-limited.
- The SendGrid webhook verifies the request signature before updating any record.
- Attachments are type- and size-checked, stored outside the web root and served only through authorized download URLs.
- Logs do not store message bodies or contact details; they reference the guardian by ID.
- Retention follows Section 8.3 and the school data-protection policy.

11.2 Non-Functional Requirements

ID	Area	Requirement
NFR-01	Timeliness	95% of absence notifications are SENT within 10 minutes of the cut-off time at a load of 300 absences.
NFR-02	Performance	Attendance write API p95 increases by no more than 50 ms; parent and staff read APIs p95 ≤ 1.5 s.
NFR-03	Reliability	No notification is lost if the worker or email provider fails; failed sends retry 3 times.
NFR-04	Availability	99.5% monthly availability during school hours (06:00–18:00).
NFR-05	Usability	Reason form usable on a 360 px wide screen; works in English and Khmer; WCAG 2.1 AA contrast.
NFR-06	Observability	Alert when failed sends exceed 5% in 15 minutes or the outbox has events older than 15 minutes.

12. Impact Analysis and Migration

Existing Area	Potential Impact	Mitigation
Attendance writes	Extra outbox insert adds load to the write path.	Single indexed insert in the same transaction; measured in staging (NFR-02).
Guardian data	Missing or wrong links mean parents are not notified.	Data-quality report before go-live; FR-11 flags missing links.
SendGrid account	Peak sending may hit rate limits or spam filters.	Confirm plan limits, set up SPF/DKIM, throttle to the account limit.
Teacher behaviour	Late corrections may cause confusing messages.	Correction window and clear correction template.
Office workflow	Staff need to adopt the follow-up list.	Short training and a two-week parallel run.

12.1 Deployment Approach

1. Deploy database migrations (new tables and STUDENT_GUARDIAN columns) – additive only.
2. Run the guardian data-quality report and fix critical gaps with the school office.
3. Deploy backend with the notification worker disabled by a feature flag.

4. Enable the worker for one pilot class, then one grade, then the whole school.
5. Release parent portal and staff screens after pilot sign-off.
6. Monitor delivery rate, time to notify and errors daily for the first two weeks.

12.2 Rollback Considerations

Rollback disables the worker feature flag and hides the new screens. Attendance taking is unaffected. New tables are kept so logs and reasons are not lost; they can be removed only after the observation period agreed with the school.

12.3 Deployment Environments and Release Gates

The same build is promoted through the environments below. Secrets such as the SendGrid key and token signing key are stored in environment configuration, never in code or this document.

Environment	Purpose	Promotion Gate
Development	Developer testing with a SendGrid sandbox.	Build passes; migrations apply cleanly; unit tests pass.
Staging / UAT	Production-like data copy; school staff test.	TC-01 to TC-18 pass; NFR-01/NFR-02 load test passes; rollback rehearsed.
Production	Live use by the school.	Pilot sign-off, approval recorded in Appendix B, monitoring alerts active.

13. Test Design

Tests cover the new behaviour and regression of attendance taking. Test data includes students with none, one and two guardians, both languages, and correction timing edge cases.

ID	Scenario	Expected Result
TC-01	Student marked Absent before cut-off	Notification sent at cut-off time, not before (FR-01).
TC-02	Student marked Absent after cut-off	Notification sent within the next worker run (FR-01).
TC-03	Student with two guardians	Each guardian receives one notification per enabled channel (FR-03).
TC-04	Late with late notifications off/on	No message when off; Late message when on (FR-02).
TC-05	Corrected to Present after 10 minutes	Exactly one correction message (FR-04).
TC-06	Corrected to Present after 45 minutes	No correction message; change is logged (FR-04).
TC-07	Worker retried / runs twice	No duplicate notifications (FR-10, unique constraint).
TC-08	Parent submits reason by link	Reason saved and visible to teacher and office (FR-05, FR-06).
TC-09	Expired or tampered token	Request rejected with 410/401; no data exposed.
TC-10	Parent resubmits reason	New reason replaces old; history kept (FR-05).
TC-11	Parent disables last channel	API returns 422; preferences unchanged (FR-09).
TC-12	No reason by 11:00	Absence appears on follow-up list (FR-07).

ID	Scenario	Expected Result
TC-13	Student without guardian	Listed as "No guardian contact" (FR-11).
TC-14	Non-admin edits template	Request denied (FR-08).
TC-15	Khmer-language parent	Khmer template used; English fallback if missing.
TC-16	SendGrid bounce webhook	Log status becomes BOUNCED; appears in follow-up (FR-10).
TC-17	Load: 300 absences at cut-off	95% sent within 10 minutes (NFR-01).
TC-18	Reason form on mobile, both languages	Usable at 360 px; contrast checks pass (NFR-05).

13.1 Regression Areas

- Teacher web attendance and QR check-in still record correctly and as fast as before.
- Existing attendance reports show the same numbers.
- Existing account emails (password reset, invitations) still send.
- Parent login and existing portal pages still work.

14. Acceptance Criteria

ID	Criterion
AC-01	F-01 to F-08 work as designed, or deviations are documented and accepted.
AC-02	TC-01 to TC-18 and the regression areas pass in Staging / UAT.
AC-03	NFR-01 and NFR-02 thresholds are met in the staging load test.
AC-04	Pilot class runs for one week with no lost or duplicate notifications.
AC-05	Guardian data-quality report reviewed and critical gaps fixed.
AC-06	Rollback rehearsed and stakeholder approval recorded before full release.

15. Requirement-to-Design Traceability

Capability	Feature IDs	Design Sections	Test Coverage
Absence & late notifications (BR-01, BR-02, BR-03)	F-01, F-02	Sections 6, 7, 8, 9	TC-01 to TC-04, TC-07, TC-17
Corrections (BR-04)	F-03	Sections 9.2, 9.3	TC-05, TC-06
Absence reasons (BR-05, BR-06)	F-04	Sections 5, 8, 9.4.2, 11	TC-08 to TC-10, TC-18
Follow-up (BR-07)	F-05	Sections 5, 9.1	TC-12, TC-13, TC-16
Channel preferences (BR-09)	F-06	Sections 5, 8, 9.1	TC-11
Admin templates & settings (BR-08)	F-07	Sections 5, 8, 9.1	TC-14, TC-15
Log & reporting (BR-10, BR-11)	F-08	Sections 8, 10	TC-07, TC-16

16. Finalized Implementation Decisions

ID	Finalized Decision	Decision Owner / Basis
OD-01	Release 1 includes F-01 to F-08; SMS/Telegram in phase 2.	BRD §6; confirm in stakeholder approval.
OD-02	Transactional outbox plus a one-minute worker, not direct sending from the API.	Reliability (NFR-03) and attendance speed (NFR-02).
OD-03	Correction window is 30 minutes from the first notification.	BRD BR-04.
OD-04	Reason links use signed tokens valid for 72 hours.	Usability for parents without portal accounts; security baseline.
OD-05	Retention: academic year + 90 days for logs and reasons; aggregates 3 years.	BRD §5 data-protection constraint.
OD-06	Default cut-off 08:30 and follow-up 11:00, configurable per school.	BRD BR-01 and BR-07.

Appendix A – Completion and Approval Checklist

- COMPLETED - Cover and document-control details filled with the design-baseline metadata.
- COMPLETED AS BASELINE - Release 1 scope defined as F-01 to F-08 in Sections 1.1 and 16.
- COMPLETED AS BASELINE - APIs, tables, fields and roles defined and traceable in Sections 8–9.
- COMPLETED AS BASELINE - Architecture, DFD, ERD, flowchart and sequence diagrams included.
- COMPLETED - Measurable performance, timeliness and retention thresholds defined.
- EXTERNAL APPROVAL REQUIRED - Reviewer names and signatures must be recorded by the school before production.

Appendix B – Review and Approval Record

This record is left for real approval. Reviewer names, dates and signatures must be supplied by the responsible people before production release.

Role	Reviewer	Decision	Date	Signature / Record Reference
Business Owner (Head of Student Affairs)	To be recorded	Pending approval	To be recorded	To be recorded
Technical Lead / Architect	To be recorded	Pending approval	To be recorded	To be recorded
Database Owner	To be recorded	Pending approval	To be recorded	To be recorded
Security / Data Protection	To be recorded	Pending approval	To be recorded	To be recorded
QA / Test Lead	To be recorded	Pending approval	To be recorded	To be recorded